

From: MPI(L) 2026 mpil2026@easychair.org
Subject: MPI(L) 2026 notification for paper 63
Date: 16 June 2026 at 00:01
To: Maria Maiorescu maria.maiorescu06@e-uvr.ro

Dear Maria Maiorescu,

The decision for your paper is accept (2.5 points).

The paper may be improved, I suggest you consider doing that.

Attached are the evaluation reports.

Best regards,
Adi

SUBMISSION: 63
TITLE: Experimental Analysis of Sorting Algorithms - An Empirical Study of Classical Sorting Algorithms Across Sizes and Input Shapes

----- REVIEW 1 -----

SUBMISSION: 63
TITLE: Experimental Analysis of Sorting Algorithms - An Empirical Study of Classical Sorting Algorithms Across Sizes and Input Shapes

----- Overall evaluation -----

SCORE: 2 (accept)

----- TEXT:

The paper is well organized, it is clearly about the subject. The author looks at six sorting algorithms and tests them on datasets of different sizes and types including random reversed and partially sorted arrays. The author does the implementation in C and the results show how the theoretical complexity classes work in life and how the input structure affects the performance of the algorithms. The results are important because the author implements algorithms from the beginning creates several types of datasets and does large-scale tests with up to one million elements. The work seems to be original in the context of this project. The discussion about Quick Sort behavior on inputs and the comparison between comparison-based algorithms and Radix Sort are particularly interesting.

The paper has enough technical details to understand and evaluate the methodology. The results seem to be correct and match the known theoretical behavior of the algorithms.

However there are some issues that need to be fixed:

- in Section 3.4 the title talks about both LSD and MSD Radix Sort but only LSD Radix Sort is actually implemented and discussed;
- the way the author measures time relies on clock() but the author should explain the difference between CPU time and real time earlier in the paper;
- there are some mistakes in the terms used such as switching between "QuickSort" and "Quick Sort";
- there are a small issues with formatting like spacing and table presentation.

----- REVIEW 2 -----

SUBMISSION: 63
TITLE: Experimental Analysis of Sorting Algorithms - An Empirical Study of Classical Sorting Algorithms Across Sizes and Input Shapes

----- Overall evaluation -----

SCORE: 2 (accept)

----- TEXT:

The paper is within the scope

The content is about how input shapes change the ranking of sorting algorithms. It is benchmarked in C, with data generators and a clock() based time layer ran over seven sized and seven shapes.

The results of this paper are reasonably important, especially since the elements extend to 10^6 which takes quadratic algorithms more than 30 mins per array. Furthermore the results are moderately difficult to achieve because, while the algorithms are textbook, the benchmark is well developed with csv logging and a repetition schedule that scales with the number of elements.

Additionally the results are known and the author states as such.

The author substantially masters the field. The only issue found with the correctness of this paper is an issue with Quicksort where at 10^4 elements has a time of 312 ms and at 10^6 has 114 ms while at 10^5 it has 12,726 ms (reversed list)

The presentation is of high standard with good structure, informative graphs and an annotated bibliography.

I recommend an accept for this paper as I find it well written, with the only recommendation for a correction being the quicksort issue

----- REVIEW 3 -----

SUBMISSION: 63
TITLE: Experimental Analysis of Sorting Algorithms - An Empirical Study of Classical Sorting Algorithms Across Sizes and Input Shapes

----- Overall evaluation -----

SCORE: 1 (weak accept)

----- TEXT:

SUMMARY OF THE PAPER

This paper investigates the gap between theoretical asymptotic complexities and actual execution times of six sorting algorithms (Bubble, Insertion, Selection, Merge, Quick, and LSD Radix Sort) implemented from scratch in C99. The author constructs a unified benchmark measuring performance across array sizes scaling from 10 to 1,000,000 integers. The study explores data layout structures by testing random, fully reversed, and partially sorted shapes containing varying percentages of structural disturbance. Key findings indicate that Radix Sort outperforms all alternatives at high scales, Insertion Sort becomes highly competitive on nearly sorted data, and structural variations significantly alter the runtime of theoretical log-linear solutions.

EVALUATION

Scope: The paper is highly relevant and fits perfectly within the scope of the conference.

The manuscript is exceptionally well-written, professionally structured, and easy to read. It systematically transitions from formal sorting theory to implementation choices, compilation parameters, and insightful multi-dimensional data plots.

The results are of high importance. Testing to scale up to an array size of 1,000,000 elements provides a true empirical view of asymptotic scaling trends. The mapping of fine-grained partially sorted states yields actionable technical insights.

While sorting algorithms are a mature area, the depth of this study—specifically evaluating multi-stage partial sorting alongside a clean, custom-built C framework—shows genuine investigative originality.

The paper is technically excellent. The timing methodology utilizing `clock()` divided by `CLOCKS_PER_SEC`, scaling down repetitions (10^6 for small arrays, 100 for medium, and single runs for large arrays) to balance execution time and statistical consistency is methodologically sound.

MINOR ERRORS AND TECHNICAL DEFICIENCIES

Quick Sort Degradation on Sorted Input - The data shows that for $n = 1,000,000$, Quick Sort encounters massive execution times on reversed input and degrades as the input becomes increasingly sorted. Although the author specifies a "median-of-three pivot" in the appendix, this behavior strongly hints at a regression toward worst-case quadratic complexity, possibly due to a subtle indexing bug in the median calculation or partition loop logic.

SUGGESTIONS FOR IMPROVEMENT

Re-verify the implementation details of the median-of-three selection. A true median-of-three approach should mitigate worst-case behavior on pre-sorted arrays, yet the data indicates severe scaling penalties on ordered layouts.

Expand Discussion on Cache Architecture. Given that the benchmark is written in native C with `-O2` optimizations, adding a brief paragraph mapping how sequential memory accesses (Radix/Insertion) vs fragmented sub-array calls (Quick Sort) influence CPU L1/L2 cache misses would elevate the final paper.

RECOMMENDATION

PROPOSAL: ACCEPT (2p)

This is a really solid study with a clean, custom-built C framework. The depth of testing across large array sizes and different shapes is great. The only catch is a weird performance drop for Quick Sort on pre-sorted arrays, which should not happen with the median-of-three pivot choice mentioned in the appendix. There is likely a small indexing bug in that partition logic. If the author can double-check the code and fix that anomaly for the final version, it will be a great paper.

